

Actividad del Módulo 3: Productización, Despliegue Avanzado y Sistemas Agénticos

Objetivo General:

Transformar el prototipo de agente conversacional (desarrollado en el Módulo 2) en una aplicación robusta de grado de producción. En este módulo final, los estudiantes deberán "productizar" su asistente integrándolo con canales de mensajería reales (WhatsApp o Telegram) para que interactúe con usuarios finales en tiempo real.

Descripción de la Actividad:

Ha llegado el momento de llevar el proyecto a la línea de meta. Para este módulo, **cada grupo deberá elegir UNA de las siguientes dos rutas arquitectónicas (Ruta A o Ruta B)** para el despliegue de su solución. Adicionalmente, se presenta un componente analítico transversal opcional (t-SNE) adaptado a cada ruta para aquellos que deseen obtener una calificación de excelencia.

ELECCIÓN DE ARQUITECTURA (Elegir Ruta A o Ruta B)



ruta A: Arquitectura Tradicional (LangChain, API REST y WhatsApp)

*Enfoque ideal para quienes desean profundizar en ingeniería de backend, orquestación manual de LLMs y flujos de automatización. En esta ruta es **obligatorio** el uso de las herramientas vistas en clase.*

1. Optimización del Agente con Técnicas Avanzadas (Function Calling):

- **Refactorización Estricta:** Los estudiantes deberán modificar el "enrutador" de herramientas implementado en el Módulo 2. En lugar de que el LLM decida de forma abierta (texto libre), implementarán **Function Calling** (o *Structured Tool Calling*).
- **Herramientas Obligatorias (LangChain):** Para evitar el uso de librerías externas no validadas o atajos, el código debe construirse estrictamente utilizando los componentes que vimos en clase. Se verificará en el repositorio de GitHub el uso de:
 - `init_chat_model` (Para inicializar el LLM).
 - `create_agent` (Para la orquestación del agente).
 - `HumanInTheLoopMiddleware` (Para control de flujos críticos).
 - `RecursiveCharacterTextSplitter` (Para el chunking de la base de conocimiento).
 - `langchain_core.vectorstores`, embeddings y vector stores nativos de LangChain.
 - `dynamic_prompt` (Para alimentar el RAG Chain).
 - `PostgresSaver` (Como checkpointer para la memoria persistente).

- **Implementación de Esquemas:** Esto implica definir un esquema estricto (JSON Schema) para cada herramienta utilizando **Pydantic**. El LLM se verá forzado a generar una salida JSON que invoca a la función correcta con los parámetros correctos, aumentando drásticamente la fiabilidad. [Revisar la documentación de LangChain sobre Structured Output](#).
- **Gestión de Errores:** El agente debe ser capaz de manejar situaciones donde una herramienta falla o no devuelve información, respondiendo de manera cortés (ej. *"En este momento no pude verificar el horario, pero puedo ayudarte con..."*).

2. "Productización" del Agente: Creación de una API REST:

- **Objetivo:** Para que un servicio externo (como N8N o WhatsApp) pueda comunicarse con el agente, este ya no puede seguir viviendo en un script de Streamlit. Debe ser expuesto como un servicio web.
- Utilizando el framework **FastAPI**, crearán un servicio web con un endpoint principal (ej. `/chat`).
- Este endpoint debe aceptar peticiones **POST**. El payload JSON debe contener el mensaje del usuario y un identificador (el número de teléfono) para mantener la memoria de la sesión con **PostgresSaver**, devolviendo la respuesta del LLM en formato JSON.

3. Integración con WhatsApp y Automatización del Flujo:

Los estudiantes deberán elegir una de las siguientes dos vías para conectar su API en FastAPI con WhatsApp:

- **Vía 1 (Enfoque Low-Code/Visual): N8N.**
 1. Configurar un proveedor de WhatsApp Business API (capa gratuita de la API Cloud de Meta o sandbox de Twilio).
 2. Crear un Webhook en N8N para recibir los mensajes entrantes.
 3. Diseñar un workflow en N8N que: se active con el webhook, realice una llamada HTTP **POST** a la API de su agente, reciba la respuesta JSON, y envíe esa respuesta de vuelta al usuario a través del nodo de WhatsApp de N8N.
- **Vía 2 (Enfoque Code-Centric Avanzado): Servidor de Integración Propio.**
 1. Construir directamente en **FastAPI** el manejador del webhook de WhatsApp para gestionar el envío y recepción de mensajes sin intermediarios (sin N8N).

Ruta Transversal A (Opcional Avanzado): Análisis de Conversaciones con t-SNE

Para conectar con el primer tema del módulo y añadir un componente analítico de ciencia de datos, los grupos pueden implementar un sistema de logging sobre su API.

- **Tarea:** Guardar cada conversación completa procesada por su API en una base de datos. Luego, utilizando un modelo de embeddings, convertir cada conversación en un vector.

- **Visualización:** Usar **t-SNE** en un notebook de Python anexo para proyectar estos vectores de conversación en un gráfico 2D/3D.
- **Análisis:** El objetivo es visualizar clústeres. ¿Qué revelan estos clústeres? (Ej: un clúster de quejas, un clúster de preguntas sobre un producto específico, un clúster de conversaciones fallidas). Esto les permite analizar el rendimiento real de su chatbot.

RUTA B: Implementación de un Sistema Operativo Agéntico (Agent OS) con OpenFang

Enfoque ideal para quienes desean explorar el estado del arte en Sistemas Agénticos, ejecución autónoma en segundo plano e infraestructura nativa en Rust.

1. Preparación del Entorno e Instalación del Agent OS (OpenFang):

- **Despliegue Local o VPS:** Los estudiantes instalarán el kernel de OpenFang en su máquina local, WSL2 o un servidor virtual ligero (exige mínimos recursos: ~32MB de binario, 2GB RAM).
`curl -fsSL https://openfang.sh/install | sh`
- **Integración de Modelos Locales (Ollama):** Para garantizar la soberanía de los datos de la empresa, iniciarán un modelo local (ej. **Gemma-4**, **Qwen 3.5**, **Llama-3.2**) usando Ollama. Aprovechando el auto-descubrimiento de OpenFang ("Zero-Config Integration"), verificarán que el OS agéntico detecte el servicio automáticamente en el puerto **11434**.

2. Migración del Conocimiento (Knowledge Base) al Sustrato de Memoria:

- El núcleo del Módulo 1 (textos limpios y estructurados producto del web scraping de la empresa del Valle del Cauca) debe ser ingerido por OpenFang.
- **Tarea:** Utilizando el sistema de memoria de 6 capas de OpenFang, inyectarán el contexto corporativo en el **Vector Store** (almacenamiento semántico) y en el **Structured KV Store** del agente asignado. De este modo, el sistema operativo asimilará la historia, productos, y servicios de la empresa como su "identidad base" permanente.

3. Configuración de Operaciones Autónomas ("Hands System"):

- A diferencia de un chatbot tradicional reactivo, OpenFang cuenta con **Hands** (playbooks de capacidades autónomas). Los estudiantes configurarán el archivo manifiesto **HAND.toml** para activar **una** o más de las siguientes operaciones para su empresa:
 - **Opción A (Perfil Comercial - "Lead Hand"):** Configurar al agente para simular la búsqueda, calificación (scoring 0-100) y exportación de clientes potenciales para los productos de su empresa.
 - **Opción B (Perfil Analítico - "Collector Hand"):** Configurar un monitor de inteligencia competitiva, donde el agente vigile menciones o cambios en la industria de la empresa en la web de forma programada.

- **Opción C (Personalizada):** Los estudiantes agregarán aquellas capacidades que consideren más adecuadas de acuerdo a la naturaleza de las empresas en las que han estado trabajando (este es el toque auténtico que diferencia cada proyecto).
- **Prueba:** Ejecutar el comando `openfang hand activate <nombre_del_hand>` y monitorear el comportamiento autónomo desde el dashboard local (<http://127.0.0.1:4200>).

4. Despliegue en Canales de Interacción (WhatsApp / Telegram):

- Haciendo uso del "Cross-Channel Canonical Sessions" (gestión de sesiones multicanal) nativo de OpenFang, conectarán su agente corporativo a un canal de mensajería real.
- **Implementación:** Configurar el puente (bridge) nativo de OpenFang para **Telegram** (creando un bot via BotFather y obteniendo el Token) o **WhatsApp** (usando el puerto local `3009` configurado en OpenFang).
- **Prueba en Vivo:** Demostrar cómo un "cliente" puede escribirle al número/bot de la empresa y ser atendido por el agente de OpenFang, el cual recuperará información del contexto ingerido (RAG interno del OS).



Ruta Transversal B (Opcional Avanzado): Análisis de Comportamiento mediante t-SNE

Para conectar con los temas de ciencia de datos, los grupos pueden implementar este componente analítico sobre el historial de interacciones de su agente de OpenFang.

- **Extracción y Vectorización:** Como OpenFang guarda un espejo de las sesiones en archivos JSONL y utiliza SQLite FTS5, los estudiantes extraerán el historial de interacciones y convertirán las transcripciones en vectores (Embeddings).
- **Visualización:** Utilizar técnicas de reducción de dimensionalidad (**t-SNE** o **UMAP**) en un notebook de Python para proyectar los vectores en un gráfico 2D/3D.
- **Análisis:** Identificar e interpretar los clústeres visuales de las intenciones de los usuarios.



ENTREGABLES: DOCUMENTACIÓN Y SUSTENTACIÓN

1. Informe Técnico Final (Unificado)

Consolidar la documentación de todos los módulos en un único informe profesional (PDF) que contenga:

- **Problema y Solución:** Breve recuento de la necesidad de la empresa y la solución planteada.
- **Evolución Arquitectónica:** Comparar la arquitectura del Módulo 2 con la del Módulo 3.
 - *Si eligieron Ruta A:* Detallar el uso de FastAPI, Function Calling estricto (LangChain) y la justificación de usar N8N vs Servidor Propio.

- *Si eligieron Ruta B*: Explicar las ventajas del Agent OS (seguridad en WASM, gestión de RAM), detallar el **HAND.toml** elegido y cómo se inyectó la memoria corporativa.
- **Diagrama End-to-End**: Un diagrama de arquitectura claro del flujo completo (Usuario > Canal > Gateway/API/OS > LLM > Retorno).
- **Análisis t-SNE (Si aplica)**: Gráficos generados y conclusiones de los clústeres.

2. Sustentación Final y Demostración en Vivo (15 minutos)

- **Cero Diapositivas**: La presentación será 100% práctica (mostrando código, terminales, dashboards).
- **Prueba de Fuego**: El profesor o un miembro del público utilizará su propio teléfono para enviarle mensajes (vía WhatsApp o Telegram) al bot de la empresa en vivo.
- **Explicación Técnica**: El equipo deberá explicar "tras bambalinas" y en tiempo real lo que está sucediendo en la terminal/dashboard/logs mientras el agente responde.



RÚBRICA DE EVALUACIÓN FINAL

1. Arquitectura Avanzada y Lógica del Agente (30%)

- *(Para Ruta A)*: Implementación estricta de las herramientas de LangChain exigidas, uso correcto de Pydantic para Structured Outputs (Function Calling) y una API REST (FastAPI) sólida.
- *(Para Ruta B)*: Migración impecable del conocimiento corporativo a la memoria de OpenFang y correcta configuración/ejecución del playbook en el **HAND.toml**.

2. Integración y Despliegue en Canal de Mensajería (30%)

- El flujo de comunicación (N8N/WhatsApp o OpenFang Bridge/Telegram-WhatsApp) es funcional, fiable y sin latencia excesiva.
- El bot maneja correctamente la memoria a corto/largo plazo en el canal y responde consistentemente de acuerdo al contexto de la empresa del Valle del Cauca, sin alucinaciones.

3. Calidad General de la Solución y Código (20%)

- El sistema completo resuelve el problema planteado inicialmente.
- El código fuente está en un repositorio de GitHub, limpio, comentado y aplicando buenas prácticas de ingeniería de software.

4. Documentación Técnica Final (10%)

- El informe es profesional, explica claramente las decisiones de diseño (Ruta A o B) y el diagrama de arquitectura es preciso.

5. Presentación y Demostración en Vivo (10%)

- La demostración en vivo es fluida. El sistema no colapsa ante la prueba del profesor/público en tiempo real.
- El equipo demuestra dominio técnico al responder las preguntas durante la sustentación.

BONUS TRANSVERSAL: Implementación de t-SNE / UMAP (Hasta +10% sobre la nota final)

- Aplicación correcta de la técnica de reducción de dimensionalidad en un notebook de Python y análisis lógico/útil de los clústeres de intenciones encontrados en las conversaciones.